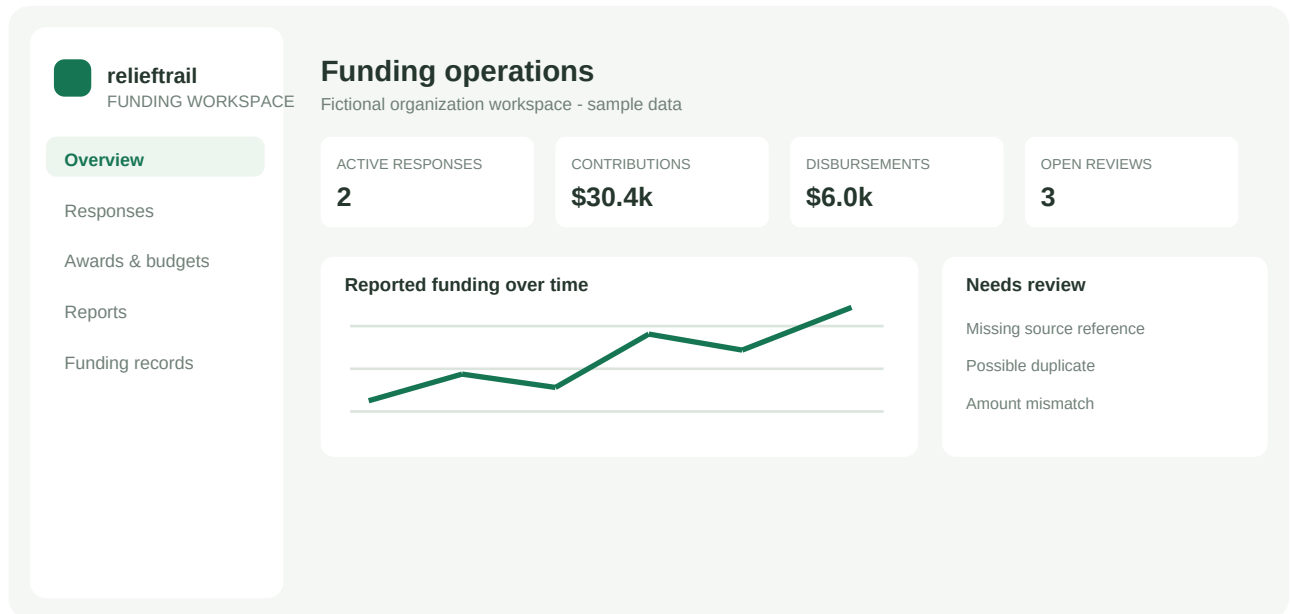


ReliefTrail ResponseHub

An organization-scoped response and grant workspace linking approved work, restricted awards, budget lines, reported spending, donor deadlines, and human review.



Illustrative interface preview based on the frontend; not a live deployment or verified database screenshot.

Project stage

Portfolio demo: React + TypeScript frontend, FastAPI backend, and PostgreSQL schema with fictional sample records. This report uses an illustrative interface preview rather than a screenshot from a live database-backed deployment.

Safety note

All data and demo credentials are fictional. The app does not accept donations, move money, or verify aid delivery. This project is not approved for real organizational data or production use.

1. Introduction

ReliefTrail ResponseHub is a database-backed portfolio application for a fictional nonprofit team. It links response records to awards and restricted allocations, lets staff record reported spending, tracks reporting milestones, and supports human review and audit history.

2. Problem statement

Funding details may arrive in separate spreadsheets and records. This makes it harder to see which response a line belongs to, spot missing references, and explain what a reviewer changed. The project demonstrates a small internal workflow that links the records and preserves the source context. It is a learning scenario, not a verified statement about a particular organization's operational pain.

3. Objectives

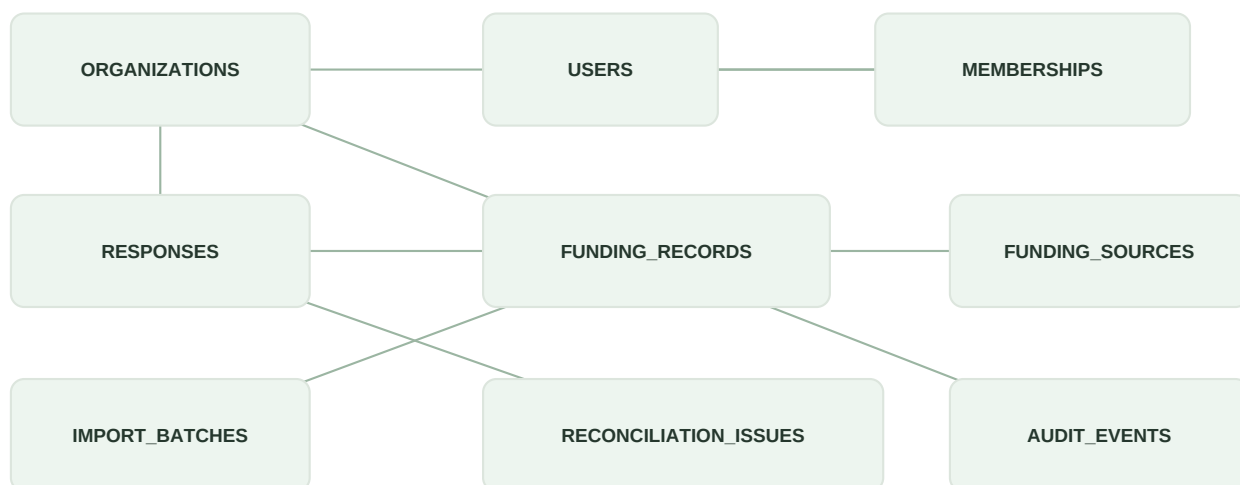
- Design normalized relational entities with primary keys, foreign keys, constraints, indexes, and organization membership.
- Build a friendly interface for response create/read/update/delete, CSV import/export, review flags, and activity history.
- Use SQL joins, aggregates, a reporting view, transaction-backed writes, and a trigger.
- Demonstrate authentication and role-aware organization access checks in a controlled fictional demo.
- Test session/CSRF behavior, role denial, CSV validation/import handling, and PostgreSQL integrity rules.

4. Technologies

Layer	Technology	Purpose
Frontend	React, TypeScript, Vite	Responsive workspace interface
Backend	Python, FastAPI, Pydantic	JSON API, validation, authorization
Database	PostgreSQL 16	Relational storage, views, trigger, transactions
Local run	Docker Compose, pnpm	Repeatable demo setup

5. ER diagram and database schema

The design separates organization, identity, response, funding, review, and audit facts. Foreign keys show how those facts connect.



Each line represents a foreign-key relationship; membership connects users to organizations.

Table	Key columns and purpose
organizations	id PK; organization name and type
users / organization_memberships	user_id + organization_id composite PK; role-based membership
responses	id PK; organization_id FK; unique response code; status, dates, summary
funders / funding_awards	Organization-scoped donor identity, award ceiling, dates, restrictions
award_allocations / reporting_milestones	Response budget line and due/submitted/accepted/returned report state
user_sessions / login_throttles	One-way session verifier/revocation; keyed failed-login throttle state
funding_sources / import_batches	Source identity and organization-scoped CSV import summaries
funding_records	response_id/source_id/import_batch_id FKs; type, amount, currency, date, references
reconciliation_issues	Response FK and optional funding-record FK; issue state
audit_events	Organization-scoped action and actor history; append-only trigger

Normalization

Related facts are stored once and connected with keys: response rows refer to organizations; funding rows refer to responses and sources. The reporting view calculates totals from funding rows instead of copying totals into response rows. This reduces duplicate data and update anomalies, following a practical third-normal-form style.

6. SQL queries and implemented features

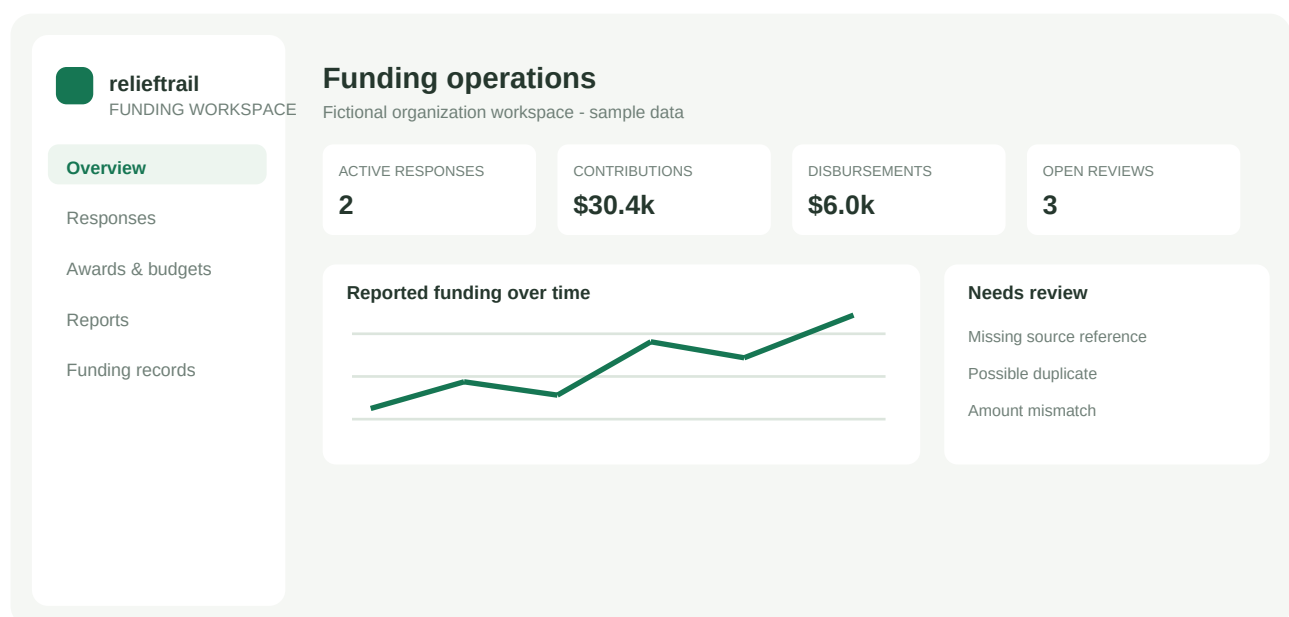
- Parameterized SELECT queries join responses, organizations, funding sources, and review issues.
- Aggregate queries calculate contribution, disbursement, response, and issue summaries within the user's organization memberships.
- The response_funding_summary view derives response totals from funding records.
- Foreign keys, uniqueness rules, and CHECK constraints protect allowed statuses, roles, currency, positive amounts, and response codes.
- CSV validation completes before database writes; accepted rows and the import batch/audit row commit in one transaction. Duplicate source references are skipped.
- An append-only PostgreSQL trigger rejects UPDATE and DELETE on audit_events.
- Argon2id password hashes, opaque revocable HttpOnly cookie sessions, CSRF checks, login throttling, and exact CORS origins protect the demo sign-in boundary.
- The API reloads active-account and organization membership state and checks roles server-side. Review responses, awards, team roles, reports, and flags by organization scope.
- Award creation groups award, allocation, reporting milestone, and audit event in one transaction. PostgreSQL triggers enforce award and disbursement ceilings.
- Only unused draft responses can be deleted; a funding or review history blocks deletion. The database trigger rejects changes to audit events.

Example relationship query

```
SELECT r.code, o.name AS organization, f.record_type, f.amount, s.name AS source
FROM funding_records f
JOIN responses r ON r.id = f.response_id
JOIN organizations o ON o.id = r.organization_id
JOIN funding_sources s ON s.id = f.source_id;
```

7. Interface preview

The preview summarizes the dashboard style and is illustrative. The frontend also includes dedicated response, awards/budgets, reporting calendar, funding, review queue, team access, and activity screens. It is not proof of a deployed service.



Illustrative interface preview based on the frontend; not a live deployment or verified database screenshot.

8. Conclusion

This project demonstrates an end-to-end database application: a typed frontend calls an API, the API validates requests and enforces tenant roles, and PostgreSQL protects linked grant records with constraints, a reporting view, triggers, and transactional writes. It is production-minded portfolio work, not certified or suitable for real operations.

9. Future scope

- Add invited-account onboarding, recovery, MFA, independent authorization review, and production operations controls.
- Add schema migration tooling, monitoring, tested backups, an incident process, and independent security review.
- Improve reconciliation rules, spreadsheet column mapping, and correction workflow.
- Explore human-reviewed AI suggestions for mapping columns or drafting reports with source-row citations.
- Explore connecting the separate contract prototype as a read-only event source after validating provenance and integrity; the projects are not currently integrated.
- Interview NGO grants and finance staff before making adoption or commercial claims.

Repository and setup

The repository includes source code, SQL schema and sample data, query examples, security and architecture docs, setup instructions, a CI workflow, and backend tests. Local API docs are available at /docs in demo mode. Local checks currently pass 14 tests; three PostgreSQL integration checks require a disposable database and are skipped when it is unavailable.